



Elementary programming

KOALA
Kind Of Advanced Language Assistant

REDSPICE

*Ce projet consiste à réaliser un simulateur d'électronique numérique en C++.
Il paraît que c'est difficile.*

Mais en fait, oui.

- KOALAB -

This document is strictly personal and must not be diffused.





INDEX

- 01 – Preliminary
- 02 – Build method

- 03 – Digital electronic
- 04 – Chipset
- 05 – Undefined states
- 06 – The project
- 07 – Technical considerations
- 08 – Bonus
- 09 – Instructions



01 – Preliminary

You must deliver your work through the software forge. Your repository name will be **redspice**.

If you make a mistake and deliver your work in another repository, the pickup system will not be able to find it and so you will not be evaluated.

You will work in a team composed of 3 people. You can of course talk with other teams but your team **must** be author of your code. Use another team's code is **cheating**. And cheating removes all your medals. Cheat checking is automatic so you **cannot** escape it.

Your delivery **must** respect the following rules :

- It must not contain any kind of object file (*.o)
- It must not contain any buffer file (*.~, #*#)
- It must not contain your final product.

The presence of a forbidden file will stop the evaluation.



02 – Build method

You must provide a Makefile to compile your whole project. You will provide a Makefile for each component. You will provide a Makefile for the simulator itself.

Your main program will be named **redspice**.

Component files will be name **component_name.so**.



03 – Digital Electronic

There is variety of programming languages, C, Lisp, Basic, APL, Intercal... They all have their specificities and may be efficient in different ways. They mainly rely on a compiler to work. This compiler will transform the code which is written in almost human language to a more primitive form.

This primitive form is the assembly language if seen as its human readable form, or machine language if seen as the microprocessor sees it.

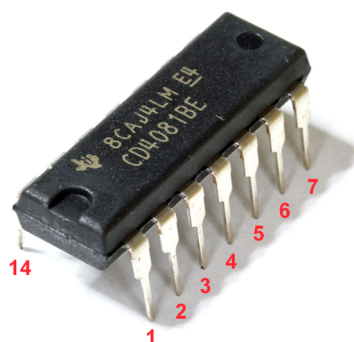
The same way there is a variety of programming languages, there is a lot of assembly languages: at least one per processor family, sometimes one per processor, sometimes several per processor.

Of course, knowing an assembly language for a microprocessor may not be enough to be efficient: a microprocessor is not a full machine. You may need to know how your machine works in order to create a usefull program in assembly: Which address for the graphic card? Which trap for this system call?

Under assembly languages and machine languages, there is the hardware itself. The hardware is built with digital electronic components: chipsets. Chipsets are tiny little functions with inputs and outputs that can be linked together to create more powerful functions, exactly like with software functions.

These functions are exclusively based on boolean logic. So it only gets and returns true or false values. In this project, you will have to create a graph. In this graph, nodes will be simulated digital electronic components, inputs or outputs.

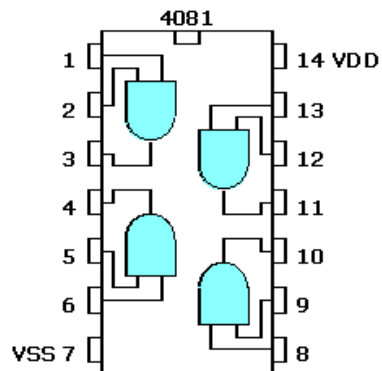
What does a chipset look like? Here is one:





04 – Chipset

Here is an example of what's inside a simple chipsets : the 4081.



The 4081 is a little box that contains four AND gates. An AND gate has two inputs (A and B) and one output (S). The 4081 features 14 connectors.

- Pin 1 and pin 2 are inputs of one of the and gate, let's say gate 1.
- Pin 3 is the output of gate 1.
- Pin 4 is the output of gate 2.
- Pin 5 and 6 are input of the gate 2.
- Pin 7 (VSS) is here for electrical purpose. We will ignore it in this project.
- Pin 8 and 9 are inputs of gate 3.
- Pin 10 is the output of gate 3.
- Pin 11 is the output of gate 4.
- Pin 12 and 13 are inputs of gate 4.
- Pin 14 (VDD) is here for electrical purpose. We will ignore it in this project.



Here is the AND gate « truth table » :

A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

Note that this is absolutely equivalent to the following function :

```
bool    and_gate(bool    a,  
                  bool    b)  
{  
    return (a && b);  
}
```

There is plenty of chipsets. Some of them are primitive, like the 4081, some others bring more complicated functions like counting or serialising functions. Microprocessors are one of the more complicated family of chipset existing by providing interpreters.

Of course, microprocessors are built with more simple chip, which happened to be built from components like the 4081 chipset. Everything can be reduced to boolean logic.



05 – Undefined states

The boolean algebra is composed of only two states : **true** and **false**. In the electronic world, **true** means VCC volt. VCC is the alimentation of the system. In a lot of case, VCC is 5 volt. In the electronic world, **false** means, in most cases, 0 volts.

It creates a problem we won't handle here : there is a lot of value under 0 and a lot above 5, and even an infinite between 0 and 5.

It also creates another problem we will have to handle: What if a component A needs to know the value S the compute its output, but S needs its output to be defined ?

The answer cannot be **true** or **false**. The answer is that the value is **undefined**, a third state. So you will not compute pure boolean algebra. You will have to integrate undefined states : one if you don't know what value is. But also another one if there is **true** and **false** mixed on the same wire : an error.

Get out bool ! Welcome **enum** !



06 – The project

REDSPICE is a logic simulator that is able to build a graph thanks to a configuration file, as well as injected values inside the graph to get results.

The Configuration File

Here is an example of configuration file. It uses a small HDL (Hardware description language). It contains a graph description.

```
#tree inputs and gate
.chipsets:
input i0
input i1
input i2
4081 and0
output out

.links:
i0:1 and0:1
i1:1 and0:2

and0:3 and0:5
i2:1 and0:6

and0:4 out:1
```

Description

The first part, which starts with the « **.chipsets:** » statement allows you to declare components that will be used by the program and name them. Components may be shared library or other configuration file. Some components may need a value. This value will be passed right after the name between parenthesis. **Input** and **output** components can be numbered with ':' followed by a digit to allow the whole file to be used as component by other configuration files.

The second part, which starts with the « **links:** » statement allows you to declare links between components. You have to precise which component you wish to link and which pin.

Space between keywords on the same line may be spaces or tabs. Their amount is irrelevant. Newline terminates the statement.



Comments start with a '#' and finish with a newline. A comment can be either at the start of a line, or after an instruction.

Here are a few special components :

- **input** : Creates a value directory linked to the command line. The value may be 0 or 1 and must be given this way : `./redspice circuit_file.rs input_name=input_value`
- **clock** : Creates an input that works the same way as input except that its value is inverted after each simulation.
- **true** : This component have one pin that is always true.
- **false** : This component have one pin that is always false.
- **output** : Creates an output that writes its state on stdout at the end of each simulation. If there are several outputs, they must be ascii sorted.

Errors

When one of the following cases happens. **REDSPICE** must raise an exception and stop the execution of the program neatly. It is forbidden to raise scalar exceptions. Moreover, exception classes must inherit from `std::exception` of the STL.

- The circuit file includes one or several lexical errors or syntactic errors.
 - A component type is unknown (Shared library or .hdl file not found).
 - A component name is unknown.
 - A given pin odes not exist.
 - Several components share the same name.
 - Not every output are linked.
 - Missing input value on command line.
 - Unknown input specified by command line.
 - No chipsets section.
 - No links section.
 - Some inputs and outputs are numbered and some others are not.

Perhaps there are others errors... However, your simulateur must never crash !



Execution

Your simulator must be able to run with a circuit file and inputs value passed as parameter. The simulator also reads on stdin a few commands.

- **exit** : it closes the program.
- **display** : it prints on stdout the value of all outputs sorted by name.
- **input=value** : it changes the value of an input (not a clock!)
- **simulate** : it launches a simulation.
- **loop** : it simulates without prompting again until SIGINT. If SIGINT is received. The prompt comes back.
- **dump** : call the Dump method of every IComponent.

Your simulator must launch one simulation directly after being launched, the display. Your simulator must handle CTRL+D correctly.

Now let's see an example of execution :

```
$> cat -e or_gate.hdl
.chipsets:
input a
input b
4071 or
output s
.links:
a:1 or:1
b:1 or:2
or:3 s:1
./redspice or_gate.hdl a=0 b=0
s=0
> a=1
> simulate
> display
s=1
> exit
$>
```



07 – Technical considerations

In order to help you with your code, we give you the following instructions that you have to respect. For each instruction, you must understand why this particular one is imposed. We are not providing these instructions randomly or to bore you to death !

The Icomponent interface

Each of your component classes UST implement the following Icomponent interface :

```
// La Caverne Aux Lapins Noirs 1999-2042
// Jason Brillante "Damdoshi"
//
// RedSpice

#ifndef __ICOMPONENT_HPP__
# define __ICOMPONENT_HPP__

namespace rs
{
    enum Tristate
    {
        BROKEN = -2,
        UNDEFINED = -1,
        FALSE = 0,
        TRUE = 1
    };

    class IComponent
    {
    public:
        /// Compute value of the precised pin
        virtual rs::Tristate Compute(size_t pin_num_this = 1) = 0;

        /// Useful to link IComponent together
        virtual void SetLink(size_t pin_num_this,
                             rs::IComponent &component,
                             size_t pin_num_target) = 0;

        /// Print on terminal the name of the component and
        /// the state of every pin of the current component
        /// The output won't be tested, but it may help you
        /// as a trace.
        virtual void Dump(void) const = 0;

        virtual ~IComponent(void) {}
    };
}

#endif // __ICOMPONENT_HPP__
```



Component classes implementing the Icomponent interface are the following ones. You can find manuals about these components in the project resources.

- 4001 : Four NOR gates.
- 4008 : 4 bits adder.
- 4011 : Four NAND gates.
- 4013 : Dual flip flop.
- 4017 : 10 bits Johnson decade.
- 4030 : Four XOR gates.
- 4040 : 12 bits binary counter.
- 4069 : Six inverter gates.
- 4071 : Four OR gates.
- 4081 : Four AND gates.
- 4094 : 8 bits shift register.
- 4514 : 4 bits decoder.
- 4801 : Random access memory.
- 2716 : Read only memory. 2716 must take a value when declared. In the .chipsets : section, you must precise a file that will contain the data written in ROM.

Example :

```
2716 rom(cp_m.dat)
```

it is FORBIDDEN to manipulate pointer or reference on each of these classes in your graph outside the initialization and the insertion. You are allowed to manipulate pointers ONLY on Icomponent.



Creation of a new IComponent

You have to write a member function of a class **relevant** to your system that will allow you to create new components in a generic way. This function must have the following prototype :

```
IComponent *CreateComponent(const std::string &type,  
                             const std::string &value) const;
```

Depending on the value of the string passed as parameter, the member function creates a new IComponent by loading a matching shared library or .hdl file.



08 – Bonus

There is several bonuses available. Here are a few example :

- Terminal output component **terminal**. Takes 8 bits as ascii code, and 1 bit which prints the character on stderr while switching from 0 to 1.
- Graphic output of the graph. (Feel free to add a `.graphic :` section inside your configuration file with graphics information) You can also use **DOT** file.
- Graphic component **7seg** : takes 4 bits as input. Display an hexadecimal number matching its entry on a graphical window.
- Graphic component **matrix** : takes 8 bits as a color shade and takes 12 bits as pixel select. Output is a 64*64 pixels picture in a graphical window.
- **i4004** or **RedSteel** support (or any other microprocessor, even your own creation. Must be turing complete)
- Stimulating a little computer.

Feel free to use SFML, SDL, Allegro or LibLapin for graphics **and for graphics only**.